SPARK HACK · NVIDIA · DGX SPARK / GB10

FlowTO — Full System Architecture

A live, 100% on-device digital twin of Toronto's road + transit network.

81,669 road edges BPR + Frank-Wolfe user equilibrium ~130 tests Built 2026-05-31

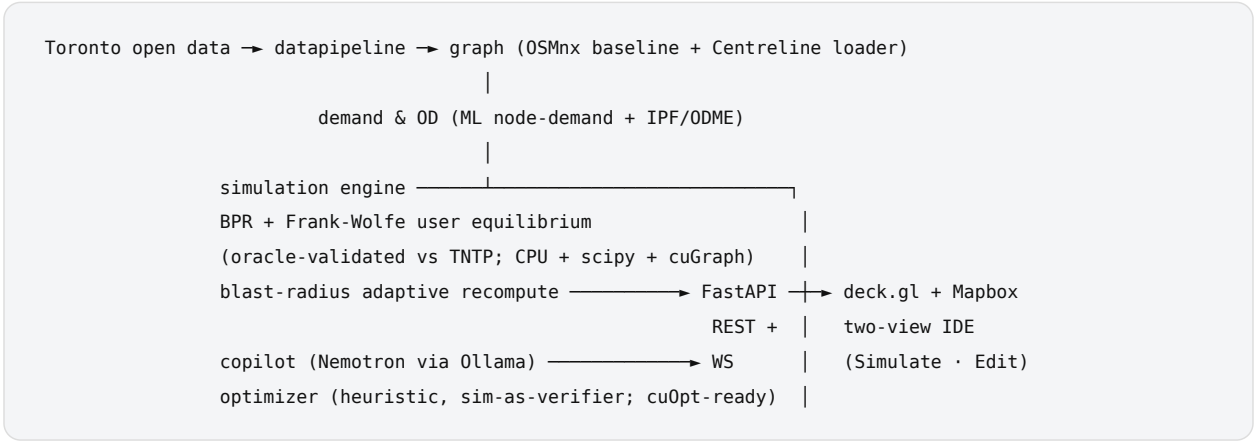
FlowTO — Full System Architecture

> A live, on-device digital twin of Toronto's road + transit network. Built for the NVIDIA Spark Hack (DGX Spark / GB10). Everything runs locally: a principled traffic-assignment engine, an adaptive blast-radius recompute, an on-device Nemotron copilot, and a two-mode deck.gl/Mapbox planner's IDE. The hero scenario is FIFA WC 2026 post-match egress at BMO Field.

This document explains the entire system end to end: how data flows in, how the graph and demand are built, how the simulation engine computes equilibrium, how the blast-radius recompute keeps edits interactive, how the API serves it, and how the frontend renders it. It is written to be read top to bottom, but each section stands alone.

Table of contents

1. The 30-second mental model



The **CPU path is the demo path**. Every number you see on the map is real engine output over the actual **81,669-edge Toronto graph** — not canned data. GPU (cuGraph, cuOpt) and the LLM (Nemotron) live behind flags; they upgrade the demo but never block it. The whole stack is **deterministic**: identical inputs always produce identical outputs, which is what makes the before/after comparisons trustworthy and the tests hermetic.

Two design rules recur everywhere and are worth internalizing up front:

- **Deterministic by construction.** No wall-clock seeding of model state, stable sort orders, fixed tie-breaks in every shortest-path backend, timestamps passed in rather than read from the clock. This is why CPU/scipy/GPU backends produce bit-for-bit identical assignments.
- **Graceful degradation.** Every accelerated or external dependency has a local fallback: GPU → scipy → CPU; XGBoost → deterministic heuristic; live Nemotron → deterministic router; real GTFS → hand-authored demo routes. The demo always runs.

---

2. Repository layout

```
flowTO/
├── src/torontosim/           # the real Python package (installable: `pip install -e .`)
│   ├── datapipeline/         # fetch (CKAN/GTFS/weather/restrictions) → Parquet + DuckDB + manifest
│   ├── graph/                # canonical road graph: OSMnx + Centreline, capacity calibration, schema
│   ├── model/                # ML node-demand → gravity OD → IPF/Furness → ODME → TTS/Census seed
│   ├── simulation/           # BPR + Frank-Wolfe UE; CPU/scipy/GPU backends; TNTP oracle
│   │   └── backends/         # cpu.py (heap Dijkstra), scipy_backend.py, gpu.py (cuGraph)
│   ├── blastradius/          # affected-OD detection + bounded cones + adaptive subgraph recompute
│   ├── optimizer/            # heuristic proposals scored by running the sim; cuOpt client
│   ├── copilot/              # Nemotron NL → validated tool calls; bylaw checker; local RAG
│   ├── transit/              # GTFS → route lines + scheduled vehicle trajectories
│   ├── perf/                 # timing harness + full-city vs blast-radius benchmark
│   ├── demo/                 # FIFA WC match-day surge + the three demo scenarios
│   └── api/                  # FastAPI: scenario CRUD, run/preview/compare, binary WS frames
├── src/{graph,model,simulation}/   # LEGACY pre-restructure copies (superseded by torontosim/*)
├── frontend/                 # React + Vite + TS; deck.gl + Mapbox + Zustand; landing + app
├── models/                   # demand_model.pkl (XGBoost) + gnn/ (GraphSAGE surrogate)
```

```
├── data/                    # graph JSON, raw/baked datasets, training CSVs, bench results
├── docs/                    # specs (00–12), research notes, this file
├── demo/                    # RUNBOOK.md + scenario JSON fixtures
├── scripts/                 # run_api.sh, run_frontend.sh, spark/* SSH harness
└── pyproject.toml           # package metadata + optional-dependency groups
```

> **Note on `src/graph`, `src/model`, `src/simulation`:** these are pre-"P00 restructure" copies that predate the consolidation into the `torontosim` package. The canonical, imported-everywhere code lives under `src/torontosim/`. When in doubt, follow the `torontosim` import path — that's what `pyproject.toml` packages and what the API, tests, and frontend talk to.

The package is configured in `pyproject.toml` with `package-dir = {"" = "src"}` and declares **optional-dependency groups** that map exactly onto the CPU/GPU/LLM gating: `dev`, `data`, `sim` (AequilibraE oracle), `model` (XGBoost), `gpu` (cudf/cugraph-cu13, Spark only), `gnn` (torch + PyG), `ai` (ollama + sentence-transformers + chromadb, Spark only), `api` (fastapi/uvicorn/pydantic), and `transit` (gtfs_kit).

---

3. The end-to-end data flow

A single trip through the system, from raw bytes to a colored road on screen:

1. **Ingest.** `datapipeline` fetches Toronto open data (Centreline, Intersections, TMC counts, signals, bridges, neighbourhoods), GTFS feeds (TTC/GO/UP), live road restrictions, and ECCC weather. It normalizes everything to Parquet + a DuckDB catalog + a SHA-256 lineage manifest.
2. **Build the graph.** `graph` turns either the committed OSMnx JSON (default, demo-safe) or the baked Centreline Parquet into a canonical NetworkX `MultiDiGraph` with per-edge capacity, speed, free-flow time, lanes, geometry, and per-field confidence labels. TMC peaks calibrate capacity upward where observed flow exceeds the model.
3. **Predict demand.** `model.predict_node_demand` runs a weather/time-aware ML model (XGBoost, or a deterministic heuristic fallback) to estimate per-intersection throughput for a given `{hour, day_of_week, month, weather}` context.
4. **Build OD.** `model.generate_od_matrix` converts node demand into an origin→destination trip table via a time-of-day-biased gravity model, optionally balanced with IPF/Furness and grounded against observed counts with ODME.
5. **Assign.** `simulation` loads OD onto the network and solves for **user equilibrium** (BPR cost function, Frank-Wolfe/Conjugate-FW), or runs the faster k-path propagation loop. Output: per-edge `load`, `pressure`, `current_time_min`, `risk`.
6. **Edit & recompute.** A planner closes a road or boosts demand. `blastradius` detects exactly which OD pairs are affected, builds a bounded subgraph around the change, and reroutes only those — interactive instead of re-solving all of Toronto.
7. **Serve.** `api` (FastAPI) exposes scenario CRUD, run/preview/compare, the WC demo, copilot, optimizer, and transit, plus a **binary WebSocket tick stream**.
8. **Render.** The frontend uploads geometry once (keyed by edge index), then streams compact 17-byte-per-edge frames and recolors the deck.gl `PathLayer` by pressure — React never touches the per-tick data.

Everything after step 2 is keyed off a single shared, read-only baseline graph held in the API's `AppState`, so all scenarios compare against the same reference.

---

4. Data pipeline

Package: `src/torontosim/datapipeline/` · **Spec:** `docs/specs/01-data-pipeline.md`

A three-verb CLI (`python -m torontosim.datapipeline {fetch,bake,verify}`) turns live City-of-Toronto open data into a verifiable offline store.

Sources & modules

MODULE	RESPONSIBILITY
`ckan.py`	City of Toronto CKAN portal client. `package_show()` , `resolve_resource(pkg, fmt, name_contains=...)` (survives resource-UUID renames), `dump_url()` , `datastore_pages()` (paginated streaming), `download_file()` (1 MB chunks, browser UA — required by City endpoints).
`gtfs.py`	Static GTFS feeds. A frozen `GtfsFeed` dataclass + `FEEDS` registry for `ttc` (via CKAN package `ttc-routes-and-schedules`), `go` and `up` (direct Metrolinx zip URLs). `fetch_feed(key, dest, date_tag=...)` .
`restrictions.py`	Live CART road-closures feed (`secure.toronto.ca/.../road_restrictions/v3`). `parse()` normalizes epoch-ms timestamps and geoPolylines into shapely `LineString` s; `fetch()` pulls live closures to simulate.
`weather.py`	ECCC hourly observations. `parse_filename()` and `fix_filenames()` repair malformed names like `weather_2023 6_00.csv` ; `categorize()` bins conditions into `clear/rain/snow/fog/storm` (with a temp ≤ −2 °C → snow fallback).
`bake.py`	Normalize raw CSV/GeoJSON → Parquet (WKT geometry, EPSG:4326 metadata) + a DuckDB catalog (one view per Parquet). Per-dataset bakers (`bake_centreline` , `bake_tmc` , …) coerce types. `verify()` checks row counts against `ROWCOUNT_FLOORS` (centreline ≥ 60k, tmc ≥ 300k, …).
`manifest.py`	Lineage. `ManifestEntry` / `Manifest` record source URL, CKAN UUID, `fetched_at` (ISO-8601, **passed in** for determinism), `sha256` , and the correct open-data license attribution per source.
`cli.py`	`fetch` (network) → `bake` (offline) → `verify` (floors). The `CKAN_DATASETS` catalog maps logical names to `(package_id, format)` . Data dir is `TS_DATA_DIR` (default `<repo>/data`).

Output layout: `data/raw/<name>.<ext>` → `data/parquet/<name>.parquet` + `data/catalog.duckdb` + `data/manifest.json` ; GTFS bakes to `data/transit/{agency}_{date}.json` .

The pipeline is split deliberately: **`fetch` is the only step that touches the network**, so `bake` / `verify` (and all tests) run hermetically offline against the raw cache.

---

5. Graph

Package: `src/torontosim/graph/` · **Specs:** `docs/specs/02-graph.md` , `docs/graph-model-and-api.md`

The canonical road network is a NetworkX **`MultiDiGraph`** (parallel edges allowed, directed). Two builders feed the *same* schema so everything downstream is source-agnostic.

Representation

- **Nodes** = intersections. Attributes: `x` / `lon` , `y` / `lat` , `name` (synthesized from incident streets, e.g. "King Street West & Spadina Avenue"), `degree` .

- **Edges** = directed street segments, keyed by a deterministic `edge_id = "{u}-{v}-{k}"`. Attributes: `road_name`, `road_class` (`motorway` / `primary` / `secondary` / `tertiary` / `residential` / `service` / `default`), `length_m`, `one_way`, `speed_kmh`, `lanes`, `capacity` (veh/h, must be > 0), `base_time_min` (free-flow), `current_time_min` (congested; `"Infinity"` when closed), `status` (`open` / `closed`), `load`, `pressure` (= load/capacity), `geometry` (`[[lat, lon], …]`), `centreline_id` (TCL join key), and a **`confidence`** dict labeling each of `{lanes, speed_kmh, capacity, one_way}` as `observed | inferred | default | manual`.

Builders

- **`build_graph.py` (OSMnx, default).** `download_graph()` pulls from OpenStreetMap; `enrich_graph()` fills speed/lanes/capacity from per-class defaults in `config.py` (`VEHICLES_PER_HOUR_PER_LANE`, `DEFAULT_SPEED_KMH`, `DEFAULT_LANES`) and computes `base_time_min = length_km / speed_kmh × 60`. Capacity = lanes × per-lane throughput.
- **`centreline_loader.py` (Toronto Centreline, opt-in).** `build_centreline_graph()` joins TCL segments to the Intersections file, maps `FEATURE_CODE_DESC` to road class (`expressway → motorway`, `major arterial → primary`, …), honors `ONEWAY_DIR_CODE` (0/1/−1) to add one or both directions, and attaches bridge clearances. Preserves `centreline_id` for TMC matching.

Supporting modules

- **`schema.py`** — single source of truth. `CANONICAL_EDGE_FIELDS`, `validate_graph()` (raises `SchemaError` on missing/null/invalid fields or capacity ≤ 0), `make_edge()`, and `ensure_confidence()` to backfill legacy graphs.
- **`calibrate_capacity.py`** — `calibrate(graph, tmc_records, min_gain=1.05)` only *raises* capacity where observed TMC peak hourly flow (max 15-min interval × 4) exceeds modeled capacity by > 5 %, and re-labels that field `observed`.
- **`mutations.py`** — in-place "what-if" ops the simulator and API share: `close_edge` (status=closed, capacity=0, time=∞, stashing prior values for reversal), `reopen_edge`, `change_capacity`, `add_edge`, `remove_edge`, `close_node`.
- **`repair.py`** — `heal_oneway_arterials()` converts unreliable OSM one-ways to two-way (default `mode="all"`, or conservative `mode="named"`) so closing a segment can't silently strand trips.
- **`routing.py`** — the `edge_id → (u, v, k)` index (`build_edge_index`), nearest-node / nearest-edge spatial lookups, `find_shortest_path`, and JSON I/O (`export_graph_json` / `import_graph_json`, which converts ∞ ↔ `"Infinity"`).
- **`config.py`** — `normalise_road_class`, `haversine_m`, `base_time_min`, plus the shared `BPR_PARAMS` (α, β per road class).

---

6. Demand & OD

Package: `src/torontosim/model/` · **Specs:** `docs/specs/03-demand-and-od.md`

This subsystem answers "how many vehicles want to go from where to where, right now?" in two stages: per-node demand, then origin→destination pairs.

Stage 1 — ML node demand

`features.py` defines a frozen 11-feature schema (`FEATURE_ORDER`) shared by training and inference so they can never drift:

`lat, lon, hour, day_of_week, month, is_weekend, weather_code, road_degree, distance_to_downtown, near_highway, road_class_rank`. Static per-node features are precomputed once per graph.

- **`train_demand_model.py`** trains a gradient-boosted regressor on `vehicle_count`. Backend is XGBoost (`device=cuda` on the Spark) or sklearn `HistGradientBoostingRegressor`. `train_with_sweep()` does a resumable randomized hyperparameter search. Synthetic data can be generated when real counts are unavailable. The artifact is a joblib payload `{model, feature_order, target, kind, metrics}` at `models/demand_model.pkl`.
- **`predict_node_demand.py`** loads that model and returns `{node_id: demand}` for a time context. Three-way fallback governed by `FLOWTO_DEMAND_MODEL`: **GNN** (if available) → **XGBoost** (the committed pkl) → **`HeuristicDemandModel`** — a deterministic closed-form formula (`base × class_factor × downtown_factor × highway_factor × degree_factor × rush_factor × weather_factor`) that mirrors the synthetic data generator exactly, so the demo never depends on a trained artifact loading.
- **`ingest_real_data.py`** builds the training CSVs from real TMC counts + weather, snapping each sensor to the nearest graph node via a cKD-tree and splitting train/val **by intersection** (no leakage).

Stage 2 — gravity OD + calibration

`generate_od_matrix.py` turns node demand into `[{origin, destination, trips}, …]`:

- **Gravity core:** `OD(i,j) = origin_strength(i) × destination_strength(j) / (1 + dist_km)`, with **time-of-day biasing** (AM peak pulls residential→downtown; PM peak pushes downtown→outer; weekend evenings add a nightlife pull). Restricted to the largest strongly-connected component, top ~1,500 origins/dests, trip lengths ∈ [0.4, 25] km, scaled to a nominal total (~100k).
- **`calibration="ipf"`** runs Furness biproportional fitting (`ipf.py`) to match production/attraction marginals; structural zeros stay zero.
- **`calibration="ipf_counts"`** additionally runs **ODME** (`odme.py`): all-or-nothing assign the seed OD to links, compute per-link observed/assigned ratios, and nudge each OD pair by the (damped) geometric mean of the ratios along its path — monotone error reduction, regularized toward the seed.

Supporting modules: `odme_calibrate.py` (node-throughput-based ODME against sensor counts), `timeofday.py` (TTS peak-hour factoring), `tts_seed.py` (survey/Census gravity prior exploded to nodes), `validate_past.py` (post-hoc predicted-vs-observed accuracy).

---

7. Simulation engine

Package: `src/torontosim/simulation/` · **Spec:** `docs/specs/04-simulation-engine.md`

The core. Given a graph and an OD table, compute the flow on every link. There are two engines and they share one congestion model and one set of pluggable shortest-path backends.

The compact network

`network.py` converts the NetworkX `MultiDiGraph` into a `Network` dataclass: parallel NumPy arrays (`tail`, `head`, `t0`, `cap`, `alpha`, `beta`) plus a **CSR adjacency** (`indptr`, `order`) built with a stable sort for determinism. This is the form every backend operates on — cache-friendly and vectorizable.

The cost function (BPR)

`bpr.py` implements the Bureau of Public Roads volume-delay function:

```
t(x) = t0 · (1 + α·(x/c)^β),   default α = 0.15, β = 4.0,   t = ∞ if c ≤ 0
```

α/β are overridable per road class (freeways tolerate higher v/c; arterials degrade sooner). It's strictly convex, which is what guarantees a unique equilibrium.

Engine A — Frank-Wolfe user equilibrium (`equilibrium.py`)

Solves Beckmann's program `min Σ ∫₀^xᵢ tᵢ(v) dv` — the state where no driver can find a faster route (Wardrop's first principle). Three variants via the `algorithm` arg: `msa` (method of successive averages), `fw` (plain Frank-Wolfe), and `cfw` **(Conjugate Frank-Wolfe, the default)** — which uses the diagonal BPR Hessian to pick a conjugate descent direction and reaches AequilibraE-grade accuracy in far fewer iterations.

Each iteration: 1. All-or-nothing load `y` on current costs (via the selected backend). 2. Relative gap `rgap = (Σ t·x − Σ t·y) / (Σ t·x)`; stop when `rgap ≤ target`. 3. Descent direction `d = y − x` (or the conjugate blend). 4. **Exact line search** by bisection on the Beckmann derivative `Σ dᵢ·tᵢ(xᵢ + λdᵢ)`. 5. `x ← x + λd`.

Initialization is a free-flow all-or-nothing load (x = 0 is infeasible). Closed/zero-cap links get infinite cost and are never chosen. `assign_equilibrium()` wraps this and writes `load` back onto the graph.

Engine B — k-path propagation loop (`assign_paths.py` + `congestion.py`)

The original, intuitive engine and the API default. For a fixed number of iterations (4): route each origin's demand over its **top-k shortest paths** (k=3), splitting demand inversely to path travel time, then recompute congestion and repeat — so each pass routes on the *previous* pass's congested times. Path diversity comes from penalizing already-used edges (`PENALTY = 3.0`). `congestion.py` maps load → `pressure = load/cap` → `current_time_min` (either the BPR formula or a legacy piecewise multiplier) → a `risk` band, applying a weather speed factor.

Pluggable backends (`backends/`)

All implement `all_or_nothing(net, costs, od_by_origin)` and produce **bit-for-bit identical** results via shared tie-break rules (sorted origins; heap ties broken by node id; equal-cost path ties broken by lowest link index, implemented as an `index × 1e-9` epsilon in the vectorized backends):

- **`cpu.py`** — heap Dijkstra over the CSR graph. Always available; the correctness anchor.
- **`scipy_backend.py`** — `scipy.sparse.csgraph.dijkstra` over collapsed parallel edges, all origins in one C call. ~30× faster than the Python heap on the Toronto graph; used for auto-calibration.
- **`gpu.py`** — cuGraph SSSP with cached topology and a virtual super-source for multi-source cones. Spark-only; falls back to CPU on any import/runtime error.

`backends/__init__.all_or_nothing()` is the dispatcher; `available_backends()` reports what's installed.

Orchestration (`simulate_traffic.py`)

`simulate_traffic(graph, od, engine=..., congestion_model=..., backend=...)` is the front door. It optionally **auto-calibrates** total demand to a target mean pressure (0.55) — exploiting that pressure is linear in demand under fixed routing, so one assignment pass sets the scale. It captures per-iteration **frames** for animation (the equilibrium engine replays its converged flow as a load ramp so the frames build deterministically to the true answer), and returns a rich dict: `summary` (assigned trips, active edges, average pressure, high-risk/severe/closed counts, stranded trips), `frames`, the mutated `graph`, and equilibrium diagnostics (`rgap`, `converged`).

`simulate_scenario(graph, od, scenario, recompute="full"|"blast")` applies mutations then re-solves — either the whole network (always correct) or just the affected subgraph (§8). `compare_simulations()` diffs two results into a summary delta + the most-impacted edges.

Validation (`oracle.py`)

The engine's correctness is anchored on the **TNTP** academic fixtures (e.g. SiouxFalls): `oracle.py` parses TNTP net/trips/flow files, the test suite solves UE, and the Conjugate-FW link flows are checked against the *published* equilibrium — plus an optional AequilibraE cross-check (import-skipped when not installed). This is "verified, not asserted."

---

8. Blast-radius

Package: `src/torontosim/blastradius/` · **Spec:** `docs/specs/05-blast-radius.md`

The problem: re-solving all of Toronto for every click is too slow to feel interactive. The insight: a local change (close one edge, throttle one corridor) only changes shortest paths in a bounded region. Blast-radius recomputes *exactly that region* and proves it equals the full recompute for the affected OD pairs.

1. **Affected-OD detection (`pathcache.py`).** Before any edit, `build_path_cache()` caches every OD pair's baseline shortest path and builds a **reverse index** `edge → set(OD)`. On a change, `PathCache.affected_ods(changed_links)` is an O(1) set lookup: exactly the OD pairs whose baseline path used a changed link.
2. **Bounded cones (`cones.py`).** `bounded_cone()` grows a cost-bounded Dijkstra **downstream** from the changed links' heads and **upstream** from their tails (default radius 8 free-flow minutes). `highway_core()` adds the endpoints of the top-capacity (90th-percentile) edges so long detours via expressways aren't missed. The union is the candidate subgraph.
3. **Reroute & widen (`recompute.py`).** `blast_assign()` reroutes only the affected OD pairs over the subgraph (`_reroute_affected`), and if boundary flow shifts by > 8 %, **adaptively widens** the radius (up to 20 min). It returns a `BlastResult` with the new per-link flow, the affected OD set, the subgraph node/link sets, and `stats` (subgraph fraction, counts) — which the frontend surfaces as "1,284 edges affected."

Both CPU (heap) and GPU (cuGraph SSSP with a virtual super-source) paths exist; GPU falls back to CPU. `tests/test_blastradius_parity.py` asserts blast output matches full recompute on the affected region.

---

9. Optimizer

Package: `src/torontosim/optimizer/` · **Spec:** `docs/specs/10-optimizer.md`

Auto-proposes interventions that improve a metric, scored by **running the actual simulation** (sim-as-verifier) rather than a learned proxy — so a proposal can never look good on paper but fail in the engine.

- **`problem.py`** — `OptimizeProblem` (objective default `average_pressure`, `budget`, `max_actions`, `candidate_k`, `capacity_multiplier`, allowed `action_space`).
- **`constraints.py`** — `mask_action()` hard-rejects illegal moves: no through-capacity boosts on residential/local streets (cut-through harm), no transit-priority corridor changes, no full motorway closure (emergency access). `ACTION_COST` + `within_budget()` enforce the capital budget.
- **`search.py`** — `greedy_search()`: take the top-k congested edges, generate capacity-uplift candidates, mask the illegal ones, **score each by calling `simulate_scenario`**, sort by improvement (tie-break by edge_id), and greedily add improving actions within budget/`max_actions`. Returns `{baseline_metric, best_metric, improvement, plan, plan_cost, candidates}` and is **never worse than baseline** (empty plan if nothing helps).

- **`cuopt_client.py`** — optional NVIDIA cuOpt client for VRP sub-problems (Spark only); raises `CuOptUnavailable` and the heuristic carries the demo.

---

10. Copilot

Package: `src/torontosim/copilot/` · **Spec:** `docs/specs/09-copilot.md`

Turns plain-language planner requests into **validated, schema-constrained tool calls**, read-only and preview-first by default, with hard bylaw guardrails and cited policy.

The flow (`planner.plan_intervention` → `plan.plan`):

1. **Deterministic router** handles rehearsed intents without any LLM — a blocked closure ("close Lakeshore both ways") returns a `refuse` with bylaw citations; the hero "ease gridlock near BMO Field" returns a pre-built mitigation plan.
2. **Live model (optional, Spark-gated).** Otherwise, POST to Ollama at `localhost:11434` running **`nemotron3:33b`** with `temperature=0`, `format=tool_call_json_schema()` (schema-constrained decoding), and a system prompt that mandates read-only-by-default and cite-your-policies.
3. **Schema validation.** The response is parsed into a Pydantic `ToolCall` (one of `preview_intervention`, `create_scenario`, `run_simulation`, `compare_scenarios`, `retrieve_policy`, `explain_edge`, `refuse`), with up to 3 re-asks on invalid JSON.
4. **Semantic checks.** Every `intervention.edge_id` must exist in `state.edge_index`; `constraints.check_request()` runs the deterministic **bylaw checker** (e.g. fully closing Lakeshore violates the fire-route code and TTC replacement-bus bylaw → returns `refuse` with `Violation`s); any state-changing call is forced to `requires_user_confirmation=True`.
5. **RAG grounding (`rag.py`).** A local TF-IDF bag-of-words retriever over a curated bylaw corpus (`copilot/corpus/*.md`) attaches the top policy docs to the response — no embeddings server required (the heavier `sentence-transformers`/`chromadb` stack is the Spark-only upgrade).

`explain.py` produces deterministic natural-language summaries of compare deltas ("Average network pressure eased by X…"). The `Intervention` schema is **shared** with the API (`api/schemas.py`), so the copilot emits exactly the objects the API validates and applies — plan → preview → confirm → apply.

---

11. Transit overlay

Package: `src/torontosim/transit/` · **Spec:** `docs/specs/08-transit-overlay.md`

A visual overlay (not part of the assignment): scheduled transit lines and moving vehicles.

- **`gtfs_reader.py`** — stdlib reader for real GTFS zips/dirs. `route_lines()` builds per-route geometry from `shapes.txt` (falling back to ordered stop coords); `trip_trajectories()` joins `stop_times` → `stops`; `build_feed_cache()` writes `data/transit/{agency}_{date}.json`; `load_cached_feed()` reads it (falling back to the newest cached date).
- **`trajectories.py`** — `build_trajectory()` produces `{path, timestamps}` with **monotonic** seconds-since-midnight (handles GTFS > 24:00 times); `interpolate_position()` linearly interpolates a vehicle's `[lng, lat]` at any time t.
- **`routes.py`** — `ROUTE_TYPE_MODE` maps GTFS route types to modes (streetcar/subway/ bus/rail/air-rail) for coloring; ships a hand-authored `DEMO_ROUTES` (509 Harbourfront, 511 Bathurst) and `demo_trajectories()` so the overlay

always renders even without a baked feed.

The API prefers cached real feeds and falls back to the demo set.

---

12. The backend API

Package: `src/torontosim/api/` · **Spec:** `docs/specs/06-backend-api.md`

FastAPI, created by `create_app(state)`. Tests inject a tiny graph; production (`serve()`) loads the full Toronto graph + baseline OD once via `_bootstrap.load_default_state()` and shares one read-only baseline.

Shared state (`store.py`)

- **`AppState`** holds the baseline `graph`, baseline `od_matrix`, the `weather` / `time_context`, a **stable `edge_ids` list** (string id → the u32 index used by binary frames) and its inverse `edge_index`, and a cached `baseline()` run (the compare reference).
- **`ScenarioStore`** is per-scenario CRUD + `run` / `preview` / `compare` orchestration over `simulate_scenario`. `edge_records(state, graph)` builds the `(idx, load, eff_speed, pressure, closed)` tuples — note effective speed is derived as `speed × base/current`.

Endpoints

ROUTE	PURPOSE
`GET /healthz`, `GET /debug/state`	health + counts
`GET /edges`	the once-uploaded edge table: `idx → edge_id + geometry/name/class`
`POST/GET/PATCH/DELETE /scenarios[/{id}]`	scenario CRUD (UUID ids, optional disk snapshots)
`POST /scenarios/{id}/run`	run with `{engine, congestion_model, backend, recompute, iterations}`; validates edge ids; returns `summary` + `blast_stats` + `rgap`
`POST /scenarios/{id}/preview`	run a hypothetical set **without** mutating stored state (asserts no scenario was added)
`GET /scenarios/{id}/compare?against=baseline`	summary delta + most-impacted edges
`GET /scenarios/{id}/records`	per-edge records of the last run (Edit-mode repaint)
`POST /copilot/plan`, `POST /copilot/explain`	copilot (501 if not installed)
`POST /optimize`	optimizer proposals (501 if not installed)
`GET /transit/{routes,trajectories}`	real cached feed → demo fallback
`GET /demo/run?scenario=baseline|wc_surge|wc_fix`	the FIFA WC demo on the real graph; deterministic → cached per scenario with a lock
`GET /jobs/{id}`	async job polling (`jobs.py` thread pool)
`WS /scenarios/{id}/stream`	binary tick frames, one per captured propagation frame, then close

The binary tick contract (`encoding.py`)

The reason the UI stays at 60 fps: geometry is uploaded **once** via `/edges`, then each edge tick is packed as a fixed 17-byte little-endian record `struct("<IfffB")` = `edge_idx:u32, load:f32, speed:f32, pressure:f32, closure:u8`, framed as `[count:u32][record × count]`. React never deserializes tick data — the typed-array store does.

Startup uses a lifespan hook that **warms only the baseline** scenario in a daemon thread (warming all three would delay the first graph response); surge and fix are computed on demand and cached.

---

13. The frontend

Package: `frontend/` · **Specs:** `docs/specs/07-frontend.md`, `docs/specs/VISUAL_TESTING.md`

React 18 + Vite + TypeScript, **two independent HTML entries** (a Vite MPA build) so the heavy map/3D code never bloats the marketing page:

- **`index.html`** → **`landing/`** — the marketing landing: Framer Motion + Lenis smooth scroll, react-three-fiber 3D scenes (CN Tower, World Cup trophy with image-based lighting), and sections (Hero, Numbers, Scenario, TwoModes, Engine, HowItWorks, CTA).
- **`app.html`** → **`main.tsx → App.tsx`** — the live planner's IDE.

The app shell

```
<TopBar/>      brand · Simulate|Edit tabs · status chip · dock toggles · theme
<ToolRail/>    Edit tools (Closure=1, Surge=2, Select=Esc)
<LeftDock/>    Saved sims (Simulate) | tool picker + scene outliner (Edit)
<MapCanvas/>   Mapbox Standard basemap + deck.gl overlay (the heart)
<BottomDock/>  timeline scrubber · playback · day-of-year picker · congestion chart
<RightDock/>   warnings ↔ details (Simulate) · Copilot chat (Nemotron)
<StatusBar/>   edges · recompute ms · subgraph edges · LLM latency · FPS · "DGX Spark · GB10"
<FirstRun/>    boot splash → "Load the twin"
```

State (Zustand)

- **`appStore.ts`** — the single UI store: theme/layout, machine state (`loaded`/`recomputing`/`status`/`planStaged`), graph + `edges`, a `pressureSeq` nonce that drives deck.gl `updateTriggers`, saved sims, Simulate selection (`selectedRoadId`), Edit state (`activeTool`, `objects`, `pendingVertices`), copilot log, timeline (`scrubberMinute`, `dayOfYear`, `playing`, `speed`), and telemetry. Actions include `loadTwin`, `applyEdits`, `placeAt`, `copilotAsk`, `applyPlan`, `selectRoad`, `recenter`, `toggleTilt`.
- **`tickStore.ts`** — module-level **typed arrays** (`load/speed/pressure/closure`) living *outside* React. `ingestFrame(buffer)` decodes a binary frame in place; `consumeDirty()` is checked once per rAF. This is the hot path — bypassing React re-renders entirely.

The map (`MapCanvas.tsx`)

Mapbox **Standard** style (time-of-day lighting via `lightPresetForMinute()` from Toronto sunrise/sunset, early 3D buildings) with a deck.gl `MapboxOverlay` interleaved in the "middle" slot. Layers: the roads `PathLayer` colored by `pressureRamp(pressure[idx])` (green→amber→red, dark-mode aware) and widened by road class; selected-road highlight; closure overlays; transit routes (mode-colored, Simulate only); the BMO Field stadium marker; pending-closure vertices; surge demand streets + directional "➤" arrows; and intervention pins. **Simulate** mode tilts to a 3D angle (pitch

52°); **Edit** mode is top-down (pitch 0°). A recompute overlay shows Demand → Assign → Pressure → Bylaw → Render progress.

Client + decoding (`api/client.ts`, `lib/decodeFrame.ts`, `api/graph.ts`)

`client.ts` wraps every REST endpoint and `connectStream(scenarioId)` opens the WebSocket; `decodeFrameInto()` mirrors the Python 17-byte format exactly. `graph.ts` reconstructs an intersection graph client-side (vertices via coordinate hashing) to power Edit-mode geometry: `corridorBetween()` (Dijkstra by road length, for closures), `streetsByDirection()` / `compassOf()` (for surge direction), and `describeSegment()` (human-readable "Road — From → To" labels, optionally upgraded via Mapbox Tilequery).

The two modes

- **Simulate** — view-only: scrub the matchday, watch egress build, click a road to chart its pressure over time, monitor warnings.
- **Edit** — place a **closure** (click two intersections → Dijkstra corridor + reverse twins → `close_edge` interventions) or a **surge** (click a street → compass radiator → `demand_surge`), which auto-calls `applyEdits()` → `POST /scenarios/{id}/run` (blast or full) → `GET …/records` → `writeRecords()` → `pressureSeq++` → recolor.

---

14. GNN surrogate

Package: `models/gnn/` · **Spec:** `docs/specs/stretch/S3-gnn-surrogate.md`

A stretch upgrade to the demand path: `model.py` defines `GraphSAGEEdgePredictor` — a multi-layer GraphSAGE node encoder feeding an edge-level head that predicts congestion pressure directly (node embeddings of both endpoints + edge features + time context). `gnn_to_sim_adapter.predict_gnn_edge_state()` returns `{edge_id: {predicted_load, predicted_pressure, predicted_time_min, risk}}` and can seed the live graph. It's gated behind the `gnn` extra (torch + PyG) and selected via `FLOWTO_DEMAND_MODEL=gnn`; any failure falls back to XGBoost, then the heuristic. Training runs on the Spark (`train_on_gx10.sh`, `requirements-gx10.txt` for the aarch64/cu13 wheels).

---

15. Performance & determinism

Package: `src/torontosim/perf/` · **Spec:** `docs/specs/11-profiling-and-perf.md`

- **`timing.py`** — near-zero-overhead instrumentation: a `timer()` context manager and `@timed` decorator append `{label, ms}` to a process-global registry; `summary()` aggregates by label. Uses `perf_counter`, never sim state.
- **`bench.py`** — `benchmark()` closes the highest-pressure edge and times `simulate_scenario(recompute="full")` vs `recompute="blast"`, reporting `speedup` and `affected_subgraph_fraction`. `to_markdown()` renders the evidence table; results land in `data/bench/`.

Determinism is enforced everywhere and is load-bearing for the before/after demo and the 130-test suite: stable sorts in the CSR build, sorted origin iteration, heap ties broken by node id, equal-cost path ties broken by lowest link index (an `index × 1e-9` epsilon in the scipy/GPU backends), `Date` / `random`-free model state, and `fetched_at` timestamps passed in rather than read from the clock.

---

16. CPU vs GPU vs LLM

The single most important architectural decision: **the CPU path is the demo path.** Accelerated and external dependencies are strictly additive.

CAPABILITY	LOCAL / CPU DEFAULT	ACCELERATED UPGRADE	FALLBACK ON FAILURE
Shortest paths / assignment	heap Dijkstra + scipy (`csgraph`)	cuGraph SSSP (`gpu` extra, Spark)	GPU → scipy → CPU, identical results
Demand model	committed XGBoost pkl	XGBoost `device=cuda`; GraphSAGE GNN	XGBoost → `HeuristicDemandModel` (closed form)
Copilot	deterministic router + bylaw checker + TF-IDF RAG	Nemotron-33B via Ollama; sentence-transformers/chromadb	live model → deterministic router
Optimizer	greedy + sim-as-verifier	cuOpt VRP client (Spark)	cuOpt → heuristic
Transit	hand-authored demo routes	real baked GTFS feeds	real feed → demo set

The Spark (GB10) GPU/LLM paths are validated over an SSH smoke harness (`scripts/spark/*`, `make spark-test`) against host `gx10-4f5f`, kept out of CI.

---

17. Build, test, deploy

```
make install          # py3.12/3.13 venv + pip install -e ".[dev,api,data]"
make test             # pytest -q -m "not spark"  → ~130 tests, CPU only
scripts/run_api.sh    # serve() loads the real graph + baseline → http://localhost:8000 (/docs)
cd frontend && npm install && npm run dev   # Vite proxies /api → :8000 → http://localhost:5173

# headless evidence
.venv/bin/python -m torontosim.demo.wc_surge --scenario all   # baseline → surge → fix
.venv/bin/python -m torontosim.perf.bench                     # full-city vs blast-radius
```

- **CI** (`.github/workflows/ci.yml`): Python 3.12, install `[dev,data,api]`, ruff + black, `pytest -m "not spark"`. GPU/AI/sim extras are intentionally excluded — the oracle test runs against committed TNTP fixtures and the AequilibraE cross-check is import-skipped.
- **Tests** (`tests/`, ~130): engine oracle + integration, blast-radius parity, IPF/ODME, determinism, API scenarios/demo/WS frames, copilot plan/RAG/constraints, data-pipeline bake, GTFS, packaging. Markers: `spark` (skipped in CI), `slow`, `network`.
- **Frontend tests** (`frontend/tests/`, Vitest): `decodeFrame`, `pressureRamp`, `tickStore`, `transit`, `redesign`.
- **Config knobs:** `TS_DATA_DIR`, `TS_GRAPH_JSON`, `TS_GRAPH_SOURCE` (`osmnx` | `centreline`), `TS_PARQUET_DIR`, `TS_MAX_PAIRS` (default 12000 OD pairs), `FLOWTO_DEMAND_MODEL` (`gnn` | `xgboost`), `VITE_API_TARGET`, `VITE_MAPBOX_TOKEN`.

---

18. Glossary

- **BPR** — Bureau of Public Roads volume-delay function `t = t0·(1 + α(v/c)^β)`; turns link volume into congested travel time.
- **User Equilibrium (UE)** — Wardrop's first principle: a flow state where no driver can unilaterally find a faster route. Computed by Frank-Wolfe on the convex Beckmann program.
- **Frank-Wolfe / Conjugate-FW** — iterative convex solvers; CFW uses the BPR Hessian for a conjugate descent direction and converges faster.
- **All-or-nothing (AON)** — load all of an OD pair's demand onto its single current shortest path; the per-iteration subproblem of Frank-Wolfe.
- **OD matrix** — origin→destination trip table; what demand the network must carry.
- **IPF / Furness** — iterative proportional fitting; balances an OD matrix to known row/column totals.
- **ODME** — OD matrix estimation; nudges a seed OD so assigned link flows match observed counts.
- **Blast-radius** — the bounded subgraph around an edit where shortest paths actually change; recomputing only it keeps edits interactive.
- **Pressure** — `load / capacity` (v/c ratio); 0 = free-flow, ≥ 1 = gridlock; the value the map colors.
- **TNTP** — Transportation Networks for Research file format; the academic fixtures (SiouxFalls, etc.) the engine is validated against.
- **CSR** — compressed-sparse-row adjacency; the cache-friendly graph layout the backends traverse.
- **Tick frame** — the 17-byte-per-edge binary record streamed over WebSocket to recolor the map without touching React. ```
